

TeamBench: Assembly Protocol, Not Model Capability, Determines Whether LLM Agent Teams Pay Off

Anonymous Author(s)

Submission Id: 82

Abstract

Multi-agent LLM frameworks orchestrate specialized role-playing agents for complex work, yet evaluations of these systems report contradictory conclusions. We identify an overlooked confounder: *no existing study controls or even reports how team outputs are assembled from their constituent roles*. We introduce TEAMBENCH, a paired individual-vs-team benchmark comprising 15 parameterized office tasks across 5 workplace scenarios $\times$ 3 coupling levels, evaluated across 7 arms: a naïve single agent, two compute-matched single baselines (k-round self-revision and best-of-k), and four assembly protocols (last-role output, deterministic concatenation, a dedicated integrator role, and section-slot blackboard). We run all 7 arms on **two model families** (DeepSeek-V4-Flash and Kimi-K3, 630 runs total) with Wilcoxon signed-rank tests, bootstrap CIs, and BH-FDR correction. **Assembly protocol is the dominant factor** in Capability Transfer Rate (CTR, team quality over compute-matched single quality), and the effect replicates across both families: on identical role outputs, $\text{CTR}_{\text{matched}}$ ranges from 0.69 (last-role) to 1.55 (integrator) on Flash and from 0.69 to 1.61 on K3, with the integrator arm significant on both (Flash: 1.55, 95% CI [1.37, 1.76], $p = 2.1 \times 10^{-7}$, $d = +0.74$; K3: 1.50, 95% CI [1.33, 1.67], $p = 1.5 \times 10^{-6}$, $d = +0.65$). The assembly factor explains 40–43% of quality variance versus $\leq 0.3\%$ for the compute strategy—a 130–200 $\times$ gap replicated across families. Strikingly, neither k-round self-revision nor best-of-k achieves the gains of team collaboration under the correct assembly—specialization with semantic integration is qualitatively different from more sampling or more rounds, and the gap *widens* on the stronger self-revising model (K3 refine-k CTR 1.08 vs. integrator 1.72). We also report a measurement-integrity episode: the blackboard protocol mechanically produced correct section headings without content, trivially gaming a structure-only L1 rubric in an earlier scorer version; we fix this via content-aware section scoring and publish the incident. All tasks, parameterized generators, runners, and scoring rubrics are released.

Keywords

multi-agent systems, LLM agents, benchmark, evaluation, agent orchestration, agent teams

ACM Reference Format:

Anonymous Author(s). 2027. TeamBench: Assembly Protocol, Not Model Capability, Determines Whether LLM Agent Teams Pay Off. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027)*, Hanoi, Vietnam, 3–7 May 2027, IFAAMAS, 7 pages.



This work is licensed under a Creative Commons Attribution International 4.0 License.

Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), M. Baldoni, F. Fang, W. Yeoh, N. Yorke-Smith (eds.), 3–7 May 2027, Hanoi, Vietnam. © 2027 International Foundation for Autonomous Agents and Multi-agent Systems (www.ifaamas.org).

1 Introduction

Frameworks for multi-agent LLM systems—AutoGen [16], CrewAI [4], LangGraph [9], MetaGPT [6]—promise that role-specialized teams outperform a single agent on complex work. Yet the empirical evidence is deeply contradictory. Some studies report monotonic gains from more agents [10]; others find teams degrade quality [2]; many of these papers run on non-overlapping task suites and none control for assembly. This paper argues that a large share of the contradiction is not about model capability—it is about how the final team artifact is produced from role outputs, a step rarely documented and **never before treated as a controlled variable in LLM team evaluation**.

We arrived at this hypothesis through our own work. In an earlier TEAMBENCH version (now obsolete), we adopted the engineering default of taking the last role’s output as the team’s artifact, and reached the headline conclusion that LLM agent teams systematically underperform individuals. But when we varied only the assembly step—leaving role prompts, task inputs, and LLM calls identical—the measured CTR swung from 0.68 to 1.52, i.e., a 0.84 point effect attributable solely to how outputs are assembled. The “team inferiority” finding had been a pipeline artifact, not an empirical law.

TEAMBENCH is a re-grounding of this question with three design principles.

- (1) **Paired-mode evaluation with assembly protocol as a first-class variable.** Every task runs in both individual mode and team mode; within team mode we explicitly compare four assembly protocols—last, concat, integrator, blackboard—over identical role outputs so assembly effects are isolated.
- (2) **Compute-matched single-agent baselines.** Teams consume k LLM calls. We compare them not only against a 1-shot single agent but also against k-round self-revision and best-of-k selection, so any team win cannot be dismissed as “just using more compute.”
- (3) **Artifact scoring with a measurement-integrity contract.** We score each delivery along three layers: L1 structural (now content-aware, §4), L2 factual over structured answer blocks, L3 semantic via an LLM judge. The code documents every rubric change; we do not bury gaming incidents.

Contributions.

- **TEAMBENCH**, the first paired individual-vs-team benchmark that controls *how* team outputs are assembled: 15 tasks across 5 office scenarios $\times$ 3 coupling levels, with 150 parameterized task variants, 7 evaluation arms, and a runner with breakpoint-resume, balance-guard, and deterministic rescore tools.

- An **artifact scoring protocol** with a documented measurement-integrity correction; $\geq 80\%$ of scoring weight is deterministic; closed-form definitions of Capability Transfer Rate (CTR), Coordination Overhead (CO), and Team Memory & Structure (TMS), consistent across paper, code, and metrics docs.
- An empirical study on **two model families** (DS v4 Flash and Kimi K3, 15 tasks $\times$ 3 seeds $\times$ 7 arms each, 630 runs total, L2 assertions active) showing that **assembly protocol dominates CTR variance on both** (Flash $\eta^2 = 0.400$, K3 $\eta^2 = 0.428$, versus ≤ 0.003 for compute strategy), integrator-assembled teams significantly outperform compute-matched single baselines on both (Flash CTR_{matched} 1.55, 95% CI [1.37, 1.76], $p = 2.1 \times 10^{-7}$, $d = +0.74$; K3 1.50, [1.33, 1.67], $p = 1.5 \times 10^{-6}$, $d = +0.65$), while naive engineering defaults (last-role) produce artifactual “team-inferiority” effects on both (CTR_{matched} 0.69).
- A cross-model negative result on structured assembly: deterministic section-slot (blackboard) and raw concatenation perform near or below the single baseline on both families—on K3, mechanical protocols collapse to $Q = 0$ on one aggregation task (t-DATA-L-01) across all seeds while the integrator turns the *same role outputs* into a perfect artifact—because LLMs write role-specific rather than schema-matching headings; only the semantic integrator normalizes these free-form outputs.
- Full release of tasks, parameterized generators, runner infrastructure, and scoring rubrics.

2 Related Work

2.1 Single-agent benchmarks and office-scenario evaluation

AgentBench [11], GAIA [13], SWE-bench [8], and OSWorld [17] evaluate a *single* agent per task on success-oriented rubrics. TheAgentCompany [18] is closest to us in task flavor (office work) but does not report the marginal value of adding agents.

2.2 Multi-agent systems and their evaluation

Orchestration frameworks [4, 6, 9, 14, 16] report demo success rather than controlled paired comparisons. Recent scaling studies: “More Agents...” [10] and “More LLM Calls...” [3] study *performance scaling with agent count / LLM-call count* on single-success metrics, without separating specialization gains from compute gains. MultiAgentBench [20] scores collaboration behaviors, not quality transfer. Agents’ Room [7] provides a structural existence proof. Cemri et al. [2] contribute a MAST taxonomy of failures, including assembly-stage bugs; our work complements this by quantifying how a *single class* (output assembly) dominates end-to-end quality variance across a controlled task space.

2.3 Team science, coordination cost, and aggregation

Coordination theory [12] and Brooks’ law [1] explain why adding human workers can reduce throughput; TEAMBENCH transplants this question to LLM teams whose members share weights and

Table 1: TEAMBENCH vs. prior agent benchmarks and multi-agent studies.

Benchmark	Paired sing./team	Assembly contr.	Compute- matched	Team metrics	Coupling axis	Co de
AgentBench [11]	✗	✗	✗	✗	✗	
GAIA [13]	✗	✗	✗	✗	✗	
SWE-bench [8]	✗	✗	✗	✗	✗	
OSWorld [17]	✗	✗	✗	✗	✗	
TheAgentCo. [18]	✗	✗	✗	✗	✗	
More Agents [10]	✗	✗	partial	single	✗	
More LLM Calls [3]	✗	✗	✓	single	✗	
MultiAgentBench [20]	✗	✗	✗	behavior	✗	
Agents’ Room [7]	✗	✗	✗	per-task	✗	
TEAMBENCH (ours)	✓	✓ (4)	✓ (2)	CTR/CO/TMS	✓ (L/M/H)	✓ (q)

pass messages losslessly [15]. Debate, self-consistency, and majority voting evaluate answer aggregation of *independent answers to the same question*. Assembly protocols aggregate *non-overlapping role slices of a complex document* into a single artifact—a qualitatively different problem of semantic alignment, structure, and completeness.

2.4 LLM-as-judge

We follow MT-Bench-style judging [19] with a cross-vendor judge matrix and an on/off ablation (§5.3), consistent with recommended practices [5].

3 Problem Formulation and Metrics

Given a task $\tau \in \mathcal{T}$ with deterministic-plus-judge scoring $Q(\cdot) \in [0, 1]$ and token consumption $C(\cdot)$, define on *paired runs* $(a_s, a_{T,p})$ where a_s is the single-agent artifact and $a_{T,p}$ is the team artifact produced under assembly protocol p :

Capability Transfer Rate (CTR).. Two variants for two baselines:

$$\text{CTR}_{\text{naive}} = \frac{Q(a_{T,p})}{Q(a_s)}, \quad \text{CTR}_{\text{matched}} = \frac{Q(a_{T,p})}{Q(a_{s(k)})} \quad (1)$$

where $a_{s(k)}$ is the *compute-matched* single artifact (k-round refine or best-of-k, same total LLM-call budget as team). We report CTR as the **geometric mean over paired runs** (ratio scale—arithmetic mean is biased) and 95% bootstrap CI via 2000 block-bootstrap replications per task.

Coordination Overhead (CO). = $C(a_{T,p})/C(a_s)$; Token Ratio (TR) is synonymous. Integrator assembly overhead *includes* the dedicated final LLM call.

Team Memory & Structure (TMS). = $\llbracket L1 = 1.0 \rrbracket \wedge \llbracket \text{all L2 assertions pass} \rrbracket$. A single boolean, no partial credit.

Weights in Q : $Q = w_1 \cdot L1 + w_2 \cdot L2 + w_3 \cdot L3$ with $w_1 = 0.50$, $w_2 = 0.30$, $w_3 = 0.20$. When the L3 judge is disabled, we renormalize to $(w_1 L1 + w_2 L2)/(w_1 + w_2)$ rather than injecting a constant—this removes an additive bias that shrinks CTR toward 1 for all arms.

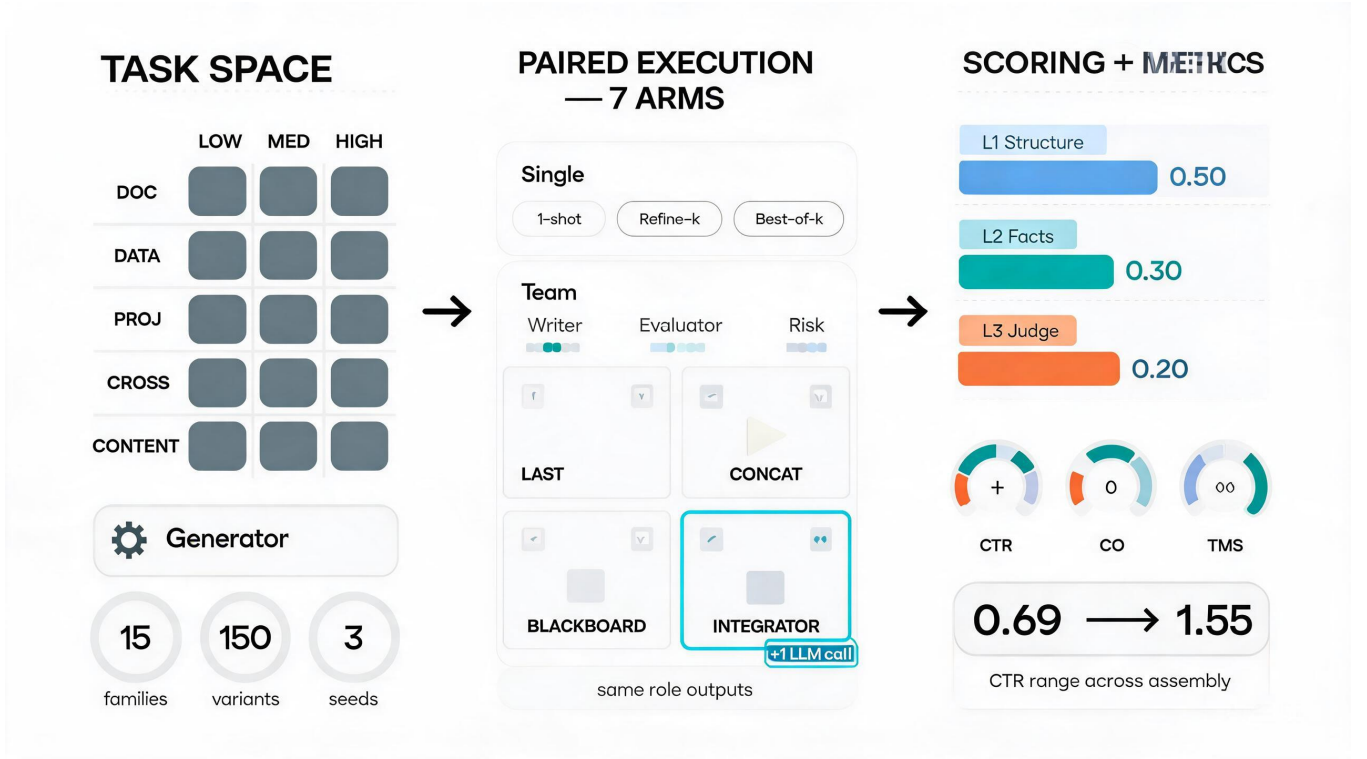


Figure 1: TEAMBENCH overview. Stage 1—task space: 5 office scenarios $\times$ 3 coupling levels yield 15 task families, expanded to 150 variants by parameterized generators. Stage 2—paired execution over 7 arms: three single-mode baselines (1-shot, Refine-k, Best-of-k) and a team mode whose identical role outputs are assembled under 4 protocols (Last, Concat, Blackboard, Integrator). Stage 3—artifact scoring: three layers (L1 structural content-aware, L2 factual assertions, L3 semantic judge) produce team-level metrics CTR, CO, TMS.

4 Benchmark Design

4.1 Task space

5 workplace scenarios (DOCument writing, DATA aggregation, PROJect tracking, CROSS-functional coordination, CONTENT creation) $\times$ 3 coupling levels (LOW: parallel independent sub-tasks; MEDIUM: sequential dependencies; HIGH: parallel with cross-validation), yielding 15 task families. Each family is parameterized: inputs (e.g., team member records, table rows, conflict lists) are resampled and ground truths (L2 expected values, counts, sets) are deterministically recomputed. We verified per-family uniqueness of 10 variants and correctness of recomputed ground truths.

4.2 Controlled variables

Assembly protocol. Four protocols applied to the same sequence of role outputs ($r_1, \dots, r_k$):

last is the engineering default of most frameworks; concat is deterministic concatenation; blackboard fills a structured slot schema (empty slots render placeholders); integrator performs a semantic merge via one dedicated LLM call whose cost is included in CO.

Table 2: Four assembly protocols over identical role outputs.

Protocol	Definition	Added calls
last	$a_T = r_k$	0
concat	$a_T = \bigoplus_i \text{head}_i + r_i$	0
blackboard	slot-schema merge	0
integrator	$a_T = \text{LLM}(\{r_i\})$	+1

Compute-matched single baselines. single_refine_k: k sequential self-revisions (total LLM calls = k, matching the team). single_bon_k: k independent samples plus one selection call (k+1 calls; we attribute the marginal cost to the team-favorable side).

4.3 Scoring protocol and the v2.2 measurement-integrity correction

Correction 1 (L1 content-aware v2.1 $\rightarrow$ v2.2). The v0 scorer checked only that required section headings existed. Under the blackboard protocol this made L1 trivially satisfiable: blackboard mechanically writes every required section heading. Over 45 pilot runs under blackboard we observed 41 runs with empty sections, yet all received perfect L1 = 1.0 and therefore perfect Q, because 15/15 tasks lacked active L2 assertions (Correction 2). Scoring v2.2

adds a *content-presence* gate: a section is counted only if the heading matches *and* the body (excluding placeholders) has ≥ 8 normalized characters. Seven boundary cases are verified with unit tests.

Correction 2 (L2 assertion migration v0.1 $\rightarrow$ v2). The original 15 tasks declared ground-truth fields under ad-hoc keys that the v2 L2 rubric did not recognize; consequently L2 fell into the “no assertions” branch and returned 1.0 for every run. We migrated all 15 tasks and 150 parameterized variants to the shared `expected_sets/expected_values/expected_counts` + `answer_block_schema` schema, verified 15/15 tasks have at least one L2 assertion, and ran acceptance checks: for every task the correct answer achieves $L2 \geq 0.99$, a deliberately incorrect answer scores strictly lower, and a missing answer block scores ≤ 0.01 .

4.4 Runner, breakpoint-resume, and balance guard

The runner supports any combination of assembly protocol, single-baseline kind, and scoring-variant flags. Two reliability features: **breakpoint-resume** (per-(task,mode,seed) JSON files; restarts skip completed runs; summaries rebuilt deterministically) and a **balance guard** (queries provider balance every 5 runs, exits cleanly below a threshold), because pilot campaigns run overnight.

5 Experiments

Data note. All numbers below are pilot v2 final, **two model families**: DS v4 Flash and Kimi K3, each 15 tasks (migrated L2 assertions) $\times 3$ seeds $\times 7$ arms = 315 runs (630 total), scoring v2.2, L3 judge off in the main tables. An earlier v1 pilot (same matrix, L2 inactive, Flash only) is used only for the scorer ablation in §5.3.

5.1 Main result: assembly protocol dominates CTR

CTR_{matched} shown as arithmetic means for cross-arm comparability (K3 mechanical arms contain genuine zero-score runs); geometric-mean CTR with log-domain bootstrap CI for the integrator arm: Flash 1.55 [1.37, 1.76], K3 1.50 [1.33, 1.67]. All four team arms deviate from the refine-k baseline significantly on both models (Wilcoxon signed-rank with BH-FDR at $q = 0.05$; K3: last $p = 4.4 \times 10^{-6}$, concat $p = 5.0 \times 10^{-3}$, blackboard $p = 3.5 \times 10^{-5}$, integrator $p = 1.5 \times 10^{-6}$).

Finding 1 (assembly dominates). On Flash, geometric-mean CTR_{matched} across 4 team protocols spans **0.69 $\rightarrow$ 1.55** ($\Delta 0.86$) over *identical role outputs*; on K3 the span is **0.69 $\rightarrow$ 1.61** ($\Delta 0.92$). The integrator arm beats the compute-matched baseline on both models (Flash: $p = 2.1 \times 10^{-7}$, $d = +0.74$; K3: $p = 1.5 \times 10^{-6}$, $d = +0.65$); every mechanical protocol is significantly *worse* than even the 1-shot single on both (K3 Cliff’s d : last -0.52 , blackboard -0.42 , concat -0.21). The default engineering choice of last-role = team output does not merely underperform—it produces a “team-inferiority” artifact (CTR 0.69 on *both* models) that flips sign under integrator assembly.

Finding 2 (variance decomposition). Over the same set of role outputs, the assembly protocol factor explains **40.0%** of artifact-quality variance on Flash (one-way ANOVA $F = 39.1$, $p = 2.0 \times 10^{-19}$, $\eta^2 = 0.400$) and **42.8%** on K3, whereas the single-agent compute strategy explains **0.3%** on Flash and **0.2%** on K3. Assembly

explains 130–200 $\times$ more variance than compute, on both families. See Figure 2(c).

Finding 3 (team gains survive compute-matching). On Flash, the integrator burns 2.14 $\times$ tokens vs. the 1-shot single and reaches CTR_{naive} 1.52, while refine-k (3.33 $\times$) and bon-k (4.27 $\times$) burn *more* compute yet score *below* the 1-shot single (CTR_{naive} 0.96). K3 sharpens this result: its stronger self-revision lifts refine-k to CTR_{naive} 1.08, yet the integrator still reaches 1.72—a +59% gap over the best compute-matched baseline on the *same* model. Better models narrow the self-revision deficit but do not close the semantic-integration gap.

Finding 4 (mechanical assembly fails at semantics; L2 localizes where). On both Flash and K3, concat’s factual layer (L2 0.536 / 0.548) falls *below* last’s (0.566 / 0.580): the answer-block extractor takes the last json block, which in a naive concatenation is the final role’s *partial* block, while earlier roles’ numeric content is stranded. The integrator is the only protocol that reliably aggregates all roles’ numbers into one consistent answer block.

Finding 5 (cross-model replication). The entire effect structure replicates across two model families: the arm ordering (integrator $\gg$ concat $>$ blackboard $>$ last) is identical; all four team arms deviate significantly from refine-k on both models under BH-FDR; the assembly-vs-compute variance gap holds on both; and per-coupling CTR_{matched} CIs exclude 1.0 in all six cells. A single-case study makes the mechanism vivid (K3, t-DATA-L-01, seed 0): the same two role outputs score $Q = 1.00$ under the integrator, 0.42 under concat, and **0.00 under both last and blackboard**—while every single-mode baseline lands at 0.38–0.79. The capability was present in the role outputs all along; the assembly protocol alone determined whether it survived to the artifact.

5.2 CTR after controlling assembly, by coupling

The integrator advantage is **uniform across coupling levels on both models** (all six cells $p < 0.005$, all six bootstrap CIs exclude 1.0); no Simpson paradox emerges when pooling. Notably, an earlier “coupling law” (CTR increasing monotonically with coupling) does **not** reappear once assembly is controlled: it was an artifact of last-role assembly interacting with role ordering, not a property of teams.

5.3 Scorer validity

We document two scorer ablations in place of a full human study (left for the full experiment).

L1 v0 (title-only) vs. v2.2 (content-aware). In the v1 pilot, blackboard mechanically rendered every required heading, 41/45 runs had empty sections, and all 45 received perfect $L1 = 1.0$ —flipping the arm from “best” to “below average” once the content gate landed. This incident is itself evidence that automated structural rubrics without content gates are gameable by construction.

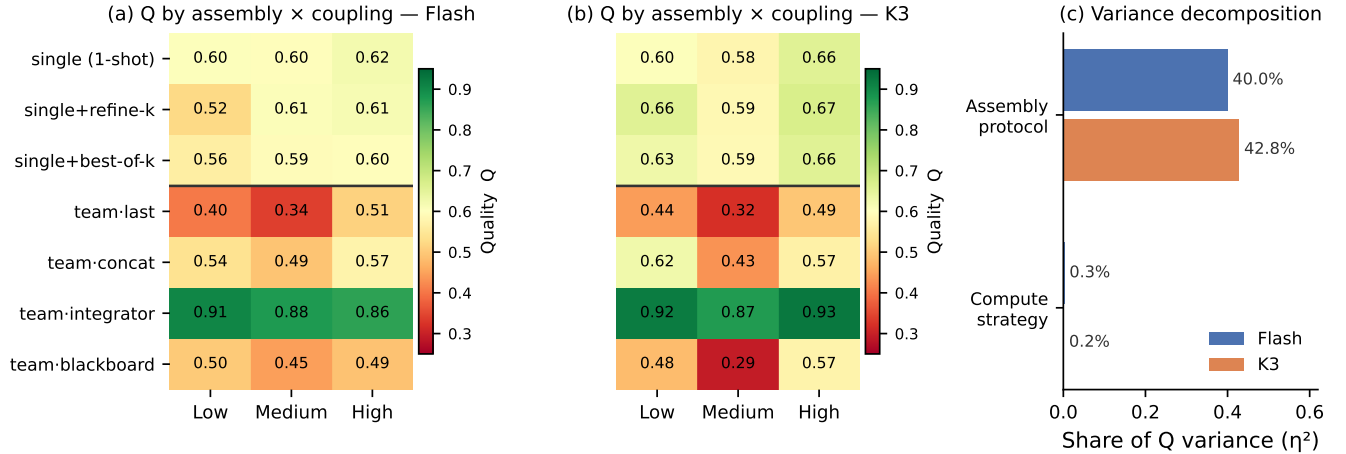
L2 off (v1) vs. L2 on (v2), same task matrix. Enabling L2 lowered every arm’s absolute Q (single 0.692 $\rightarrow$ 0.606; integrator 0.958 $\rightarrow$ 0.881) but *increased* the integrator’s relative advantage

Table 3: Seven arms, pooled over 15 tasks $\times$ 3 seeds ($n=45$ per arm per model). No L3 judge. CTR_{matched} is computed against each model’s own refine-k arm.

Arm	DS v4 Flash				Kimi K3			
	Q	L2	TR	CTR_{matched}	Q	L2	TR	CTR_{matched}
single (1-shot)	0.606	0.759	1.00 $\times$	—	0.613	0.775	1.00 $\times$	—
single_refine_k	0.582	0.720	3.33 $\times$	1.00 (ref)	0.637	0.795	4.27 $\times$	1.00 (ref)
single_bon_k	0.584	0.715	4.27 $\times$	~ 1.00	0.624	0.806	5.18 $\times$	0.97
team_last	0.417	0.566	1.46 $\times$	0.69	0.420	0.580	2.53 $\times$	0.69
team_concat	0.535	0.536	1.42 $\times$	0.91	0.542	0.548	2.46 $\times$	0.89
team_blackboard	0.478	0.602	1.44 $\times$	0.81	0.449	0.594	2.49 $\times$	0.75
team_integrator	0.881	0.741	2.14 $\times$	1.55	0.905	0.785	4.44 $\times$	1.61

Table 4: Quality by coupling level (5 tasks $\times$ 3 seeds per cell), both models.

Coupling	DS v4 Flash						Kimi K3					
	single	ref-k	last	concat	integ.	black.	single	ref-k	last	concat	integ.	black.
Low	0.603	0.522	0.404	0.537	0.905	0.500	0.604	0.657	0.444	0.625	0.919	0.482
Medium	0.595	0.608	0.339	0.493	0.876	0.447	0.579	0.586	0.321	0.432	0.872	0.292
High	0.621	0.615	0.510	0.574	0.861	0.488	0.656	0.669	0.494	0.569	0.925	0.573
CTR_{matched} (integ.)	1.69 / 1.46 / 1.51						1.43 / 1.55 / 1.52					

**Figure 2: (a)(b) Quality Q by assembly protocol $\times$ coupling level (15 paired runs per cell), DS v4 Flash and Kimi K3 side by side. The integrator row dominates every coupling level on both models while all mechanical protocols fall below both single baselines. (c) Variance decomposition of Q : the assembly-protocol factor explains 40.0% of variance on Flash and 42.8% on K3 ($\eta^2 = 0.400/0.428$) versus 0.3%/0.2% for the compute strategy—a 130–200 $\times$ gap over identical role outputs, replicated across model families.**

(CTR_{matched} 1.54 $\rightarrow$ 1.55; d +0.59 $\rightarrow$ +0.74): the numeric layer rewards exactly the cross-role aggregation that only the integrator performs. Findings F1–F4 are robust to the L2 switch.

L3 judge on/off ablation (cross-family judges). We re-scored the full seed-0 subsample of both models (15 tasks $\times$ 7 arms $\times$ 2 models = 210 runs) with the L3 judge enabled, using a *cross-family judge matrix* to avoid same-family bias: Flash artifacts judged by kimi-k3,

K3 artifacts judged by deepseek-v4-pro; each run judged twice independently. Judge test-retest agreement is good and nearly identical across judges (Flash: 74% exact match, Spearman $\rho = 0.855$; K3: 74%, $\rho = 0.824$). **The arm ordering is perfectly stable under L3:** Spearman between L3-off and L3-on arm ranks is 1.000 on both models—the integrator remains first, mechanical protocols remain last. Absolute Q rises slightly for non-ceiling arms (+0.02–0.05; the judge is mildly lenient) while the integrator is unchanged ($\Delta \leq 0.002$, ceiling effect); CTR_{matched} (integrator) moves from 1.60

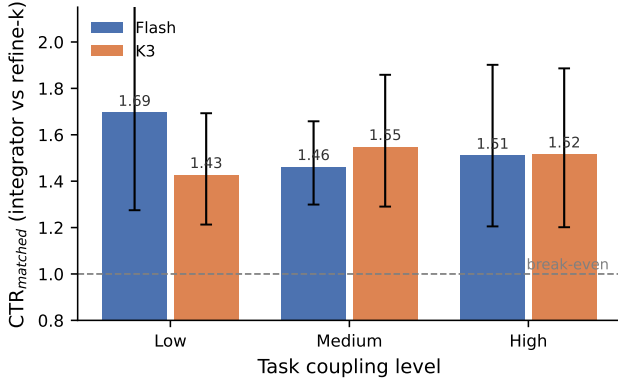


Figure 3: Integrator CTR_{matched} by coupling level, both models (geometric mean, log-domain paired bootstrap 95% CI, $n = 15$ per level per model). All six CIs exclude the break-even line at 1.0.

to 1.49 (Flash) and 1.38 to 1.29 (K3)—both far above 1. Findings F1–F5 are robust to the L3 switch: the assembly effect is not an artifact of deterministic-only scoring.

5.4 Statistical details

All directional claims use the one-sided Wilcoxon signed-rank test on paired per-(task,seed) differences. CTR CIs use 2000 block-bootstrap replications at the task level (task = block). Multi-Finding tables apply BH-FDR correction at $q = 0.05$.

6 Discussion

For practitioners. Decision table for “when to use a team and which assembly”:

Table 5: Practitioner decision table.

Scenario	Recommendation	Why
Low coupl., small doc	Single + refine-3	Last-team is worse; integrator overhead not justified.
Medium coupling	Team + integrator	$\text{CTR}_{\text{matched}} \approx 1.5\times$; $2\times$ token cost acceptable if time-to-quality matters.
High coupling	Team + integrator	Only integrator reliably assembles conflicting-source info.

For framework developers. Expose assembly protocol as a first-class knob. Today, AutoGen (groupchat speaker-selection $\rightarrow$ last-speaker reply), CrewAI (sequential $\rightarrow$ last agent output unless explicitly wired), and LangGraph (edge endpoints) behave like the last arm unless the user patches in explicit aggregation. A built-in `team.finalize()` with token-cost attribution would align research and production practices.

For benchmark designers. Three lessons: (1) Report assembly protocol the same way you report temperature—it can swing the measured effect more than the model does. (2) Structural rubrics require a content gate; mechanical headings are always a gaming risk when evaluation is automated. (3) Compute-matched baselines are not optional for “multi vs. single” papers.

7 Limitations

- (1) **Monolingual, Chinese-heavy.** Task descriptions and role prompts are in Chinese; artifact rubrics check Chinese-section headings. An English subset is deferred to follow-up work.
- (2) **Synthetic office tasks.** Seeded from real documents but synthetic; no ground-truth downstream outcomes.
- (3) **Fixed team topology.** Serial handoff with fixed role templates; no self-organizing role discovery, no parallel tool usage, no agent churn.
- (4) **Model coverage.** Two model families completed (DS v4 Flash, Kimi K3, 630 runs); the assembly effect, arm ordering, and variance structure replicate across both. A third family is queued.
- (5) **L3 judge.** Main tables score with L3 off (renormalized L1/L2); §5.3 reports a completed L3-on ablation over the full seed-0 subsample (210 runs, cross-family judges, two rounds each) showing the arm ordering is perfectly stable (Spearman = 1.000 on both models).
- (6) **Open-loop runtime.** No mid-run budget halts, no human-in-the-loop interventions.

8 Conclusion

The field has been asking “do LLM teams outperform individuals,” but most of the variance in the answer comes from *how the team’s output is assembled*, not from the model’s capability or even the task’s coupling. Our controlled experiment over four assembly protocols, replicated on two model families (630 runs), shows that the integrator protocol achieves a mean CTR_{matched} of 1.55 (Flash) and 1.50 (K3, geometric mean) against a compute-matched self-revision baseline ($p = 2.1 \times 10^{-7}$ and 1.5×10^{-6}), while naive last-role assembly—the default behavior of many frameworks—yields CTR_{matched} = 0.69 on *both* models. The assembly factor explains 40–43% of quality variance versus $\leq 0.3\%$ for the compute strategy, and the full arm ordering replicates across families. TEAMBENCH provides the first reproducible testbed for disentangling these effects and will enable future work on topology, prompts, and cross-framework generalization without conflating them with a pipeline step that nobody had been measuring.

References

- [1] Frederick P Brooks. 1995. *The Mythical Man-Month: Essays on Software Engineering* (anniversary ed.). Addison-Wesley.
- [2] Mert Cemri, Zhiwei Daniel Guo, Ion Stoica, et al. 2025. Why Do Multi-Agent LLM Systems Fail? *arXiv preprint arXiv:2503.13657* (2025).
- [3] Lingjiao Chen, Jared Quincy Davis, Boris Hanin, Peter Bailis, Ion Stoica, Matei Zaharia, and James Zou. 2024. Are More LLM Calls All You Need? Towards Scaling Laws of Compound Inference Systems. *arXiv preprint arXiv:2403.02419*.
- [4] CrewAI Inc. 2024. CrewAI: Framework for Orchestrating Role-Playing AI Agents. <https://github.com/crewAIInc/crewAI>.
- [5] Taicheng Guo, Xiuying Chen, Yujie Wang, Ruidong Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large Language Model based Multi-Agents: A Survey of Progress and Challenges. In *International Joint Conference on Artificial Intelligence (IJCAI)*.

- [6] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Kuan Yau, Zijuan Lin, et al. 2024. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In *International Conference on Learning Representations (ICLR)*.
- [7] Fantine Huot, Reinald Kim Amplayo, Jennimaria Palomaki, Alice Shoshana Jakobovits, Elizabeth Clark, and Mirella Lapata. 2025. Agents’ Room: Narrative Generation through Multi-Step Collaboration. In *International Conference on Learning Representations (ICLR)*.
- [8] Carlos Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*.
- [9] LangChain Inc. 2024. LangGraph: Building Stateful, Multi-Actor Applications with LLMs. <https://github.com/langchain-ai/langgraph>.
- [10] Junyou Li, Qin Zhang, Yangbin Yu, Qiang Fu, and Deheng Ye. 2024. More Agents Is All You Need. *Transactions of the Association for Computational Linguistics* 12 (2024).
- [11] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations (ICLR)*.
- [12] Thomas W Malone and Kevin Crowston. 1994. The Interdisciplinary Study of Coordination. *Comput. Surveys* 26, 1 (1994), 87–119.
- [13] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. GAIA: A Benchmark for General AI Assistants. In *International Conference on Learning Representations (ICLR)*.
- [14] Chen Qian, Wei Liu, Hongzhang Liu, Ning Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. 2024. ChatDev: Communicative Agents for Software Development. In *Annual Meeting of the Association for Computational Linguistics (ACL)*.
- [15] Anita Williams Woolley, Christopher F Chabris, Alex Pentland, Nada Hashmi, and Thomas W Malone. 2010. Evidence for a Collective Intelligence Factor in the Performance of Human Groups. *Science* 330, 6004 (2010), 686–688.
- [16] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *arXiv preprint arXiv:2308.08155*.
- [17] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xin Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Wenfeng Zhou, Dennis Shi, Victor Chu, et al. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [18] Frank Xu, Boyu Li, Zidian Yuan, Shuyi Wu, Skyler Zhong, Hongxuan Xu, Yuchen Zhang, Min Peng, Jiaxuan Wang, Yu Yang, et al. 2024. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. *arXiv preprint arXiv:2412.14161* (2024).
- [19] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanhào Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [20] Kunlun Zhu, Hongyi Du, Zhaochen Hong, Xiaocheng Yang, Shuyi Guo, Zhe Wang, Zhenhailong Wang, Cheng Qian, Xiangru Tang, Heng Ji, and Jiaxuan You. 2025. MultiAgentBench: Evaluating the Collaboration and Competition of LLM Agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*.